Use the light sensor

Article • 05/06/2023

Learn how to use the ambient light sensor to detect changes in lighting.

Important APIs

- Windows.Devices.Sensors
- LightSensor

Prerequisites

You should be familiar with Extensible Application Markup Language (XAML), Microsoft Visual C#, and events.

The device or emulator that you're using must support an ambient light sensor.

Create a simple light-sensor app

An ambient light sensor is one of the several types of environmental sensors that allow apps to respond to changes in the user's environment.

> **Note**
>
> For a more complete implementation, see the **light sensor sample**.

Instructions

- Create a new project, choosing a **Blank App (Universal Windows)** from the **Visual C#** project templates.
- Open your project's BlankPage.xaml.cs file and replace the existing code with the following.

C#

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using Windows.Foundation;
using Windows.Foundation.Collections;
```

```
    using Windows.UI.Xaml;
    using Windows.UI.Xaml.Controls;
    using Windows.UI.Xaml.Controls.Primitives;
    using Windows.UI.Xaml.Data;
    using Windows.UI.Xaml.Input;
    using Windows.UI.Xaml.Media;
    using Windows.UI.Xaml.Navigation;

    using Windows.UI.Core; // Required to access the core dispatcher object
    using Windows.Devices.Sensors; // Required to access the sensor platform
and the ALS

    // The Blank Page item template is documented at
https://go.microsoft.com/fwlink/p/?linkid=234238

    namespace App1
    {
        /// <summary>
        /// An empty page that can be used on its own or navigated to within
a Frame.
        /// </summary>
        public sealed partial class BlankPage : Page
        {
            private LightSensor _lightsensor; // Our app' s lightsensor
object

            // This event handler writes the current light-sensor reading to
            // the textbox named "txtLUX" on the app' s main page.

            private void ReadingChanged(object sender,
LightSensorReadingChangedEventArgs e)
            {
                Dispatcher.RunAsync(CoreDispatcherPriority.Normal, (s, a) =>
                {
                    LightSensorReading reading = (a.Context as
LightSensorReadingChangedEventArgs).Reading;
                    txtLuxValue.Text = String.Format("{0,5:0.00}",
reading.IlluminanceInLux);
                });
            }

            public BlankPage()
            {
                InitializeComponent();
                _lightsensor = LightSensor.GetDefault(); // Get the default
light sensor object

                // Assign an event handler for the ALS reading-changed event
                if (_lightsensor != null)
                {
                    // Establish the report interval for all scenarios
                    uint minReportInterval =
_lightsensor.MinimumReportInterval;
                    uint reportInterval = minReportInterval > 16 ?
minReportInterval : 16;
```

```
                    _lightsensor.ReportInterval = reportInterval;

                    // Establish the even thandler
                    _lightsensor.ReadingChanged += new
TypedEventHandler<LightSensor, LightSensorReadingChangedEventArgs>
(ReadingChanged);
                }

            }

        }
    }
```

You'll need to rename the namespace in the previous snippet with the name you gave your project. For example, if you created a project named **LightingCS**, you'd replace `namespace App1` with `namespace LightingCS`.

- Open the file MainPage.xaml and replace the original contents with the following XML.

XML

```
    <Page
        x:Class="App1.BlankPage"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="using:App1"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
        mc:Ignorable="d">

        <Grid x:Name="LayoutRoot" Background="Black">
            <TextBlock HorizontalAlignment="Left" Height="44"
Margin="52,38,0,0" TextWrapping="Wrap" Text="LUX Reading"
VerticalAlignment="Top" Width="150"/>
            <TextBlock x:Name="txtLuxValue" HorizontalAlignment="Left"
Height="44" Margin="224,38,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="217"/>


        </Grid>

    </Page>
```

You'll need to replace the first part of the class name in the previous snippet with the namespace of your app. For example, if you created a project named **LightingCS**, you'd replace `x:Class="App1.MainPage"` with `x:Class="LightingCS.MainPage"`. You should also replace `xmlns:local="using:App1"` with `xmlns:local="using:LightingCS"`.

- Press F5 or select **Debug** > **Start Debugging** to build, deploy, and run the app.

Once the app is running, you can change the light sensor values by altering the light available to the sensor or using the emulator tools.

- Stop the app by returning to Visual Studio and pressing Shift+F5 or select **Debug** > **Stop Debugging** to stop the app.

Explanation

The previous example demonstrates how little code you'll need to write in order to integrate light-sensor input in your app.

The app establishes a connection with the default sensor in the **BlankPage** method.

C#

```
_lightsensor = LightSensor.GetDefault(); // Get the default light sensor
object
```

The app establishes the report interval within the **BlankPage** method. This code retrieves the minimum interval supported by the device and compares it to a requested interval of 16 milliseconds (which approximates a 60-Hz refresh rate). If the minimum supported interval is greater than the requested interval, the code sets the value to the minimum. Otherwise, it sets the value to the requested interval.

C#

```
uint minReportInterval = _lightsensor.MinimumReportInterval;
uint reportInterval = minReportInterval > 16 ? minReportInterval : 16;
_lightsensor.ReportInterval = reportInterval;
```

The new light-sensor data is captured in the **ReadingChanged** method. Each time the sensor driver receives new data from the sensor, it passes the value to your app using this event handler. The app registers this event handler on the following line.

C#

```
_lightsensor.ReadingChanged += new TypedEventHandler<LightSensor,
LightSensorReadingChangedEventArgs>(ReadingChanged);
```

These new values are written to a TextBlock found in the project's XAML.

XML

```
<TextBlock HorizontalAlignment="Left" Height="44" Margin="52,38,0,0"
TextWrapping="Wrap" Text="LUX Reading" VerticalAlignment="Top" Width="150"/>
 <TextBlock x:Name="txtLuxValue" HorizontalAlignment="Left" Height="44"
Margin="224,38,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="217"/>
```

Use the orientation sensor

Article • 05/06/2023

Learn how to use the orientation sensors to determine the device orientation.

Important APIs

- Windows.Devices.Sensors
- OrientationSensor
- SimpleOrientation

Prerequisites

You should be familiar with Extensible Application Markup Language (XAML), Microsoft Visual C#, and events.

The device or emulator that you're using must support an orientation sensor.

Create an OrientationSensor app

An orientation sensor is one of the several types of environmental sensors that allow apps to respond to changes in the device orientation.

There are two different types of orientation sensor APIs included in the Windows.Devices.Sensors namespace: OrientationSensor and SimpleOrientation. While both of these sensors are orientation sensors, that term is overloaded and they are used for very different purposes. However, since both are orientation sensors, they are both covered in this article.

The OrientationSensor API is used for 3-D apps two obtain a quaternion and a rotation matrix. A quaternion can be most easily understood as a rotation of a point [x,y,z] about an arbitrary axis (contrasted with a rotation matrix, which represents rotations around three axes). The mathematics behind quaternions is fairly exotic in that it involves the geometric properties of complex numbers and mathematical properties of imaginary numbers, but working with them is simple, and frameworks like DirectX support them. A complex 3-D app can use the Orientation sensor to adjust the user's perspective. This sensor combines input from the accelerometer, gyrometer, and compass.

The SimpleOrientation API is used to determine the current device orientation in terms of definitions like portrait up, portrait down, landscape left, and landscape right. It can also detect if a device is face-up or face-down. Rather than returning properties like

"portrait up" or "landscape left", this sensor returns a rotation value: "Not rotated", "Rotated90DegreesCounterclockwise", and so on. The following table maps common orientation properties to the corresponding sensor reading.

Expand table

Orientation	Corresponding sensor reading
Portrait Up	NotRotated
Landscape Left	Rotated90DegreesCounterclockwise
Portrait Down	Rotated180DegreesCounterclockwise
Landscape Right	Rotated270DegreesCounterclockwise

> **Note**
>
> For a more complete implementation, see:
>
> - Orientation sensor sample
> - Simple orientation sensor sample

Instructions

- Create a new project, choosing a **Blank App (Universal Windows)** from the **Visual C#** project templates.
- Open your project's MainPage.xaml.cs file and replace the existing code with the following.

C#

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using Windows.Foundation;
using Windows.Foundation.Collections;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.UI.Xaml.Controls.Primitives;
using Windows.UI.Xaml.Data;
using Windows.UI.Xaml.Input;
using Windows.UI.Xaml.Media;
using Windows.UI.Xaml.Navigation;
```

```
    using Windows.UI.Core;
    using Windows.Devices.Sensors;

    // The Blank Page item template is documented at
https://go.microsoft.com/fwlink/p/?linkid=234238

    namespace App1
    {
        /// <summary>
        /// An empty page that can be used on its own or navigated to within
a Frame.
        /// </summary>
        public sealed partial class MainPage : Page
        {
            private OrientationSensor _sensor;

            private async void ReadingChanged(object sender,
OrientationSensorReadingChangedEventArgs e)
            {
                await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, ()
=>
                {
                    OrientationSensorReading reading = e.Reading;

                    // Quaternion values
                    txtQuaternionX.Text = String.Format("{0,8:0.00000}",
reading.Quaternion.X);
                    txtQuaternionY.Text = String.Format("{0,8:0.00000}",
reading.Quaternion.Y);
                    txtQuaternionZ.Text = String.Format("{0,8:0.00000}",
reading.Quaternion.Z);
                    txtQuaternionW.Text = String.Format("{0,8:0.00000}",
reading.Quaternion.W);

                    // Rotation Matrix values
                    txtM11.Text = String.Format("{0,8:0.00000}",
reading.RotationMatrix.M11);
                    txtM12.Text = String.Format("{0,8:0.00000}",
reading.RotationMatrix.M12);
                    txtM13.Text = String.Format("{0,8:0.00000}",
reading.RotationMatrix.M13);
                    txtM21.Text = String.Format("{0,8:0.00000}",
reading.RotationMatrix.M21);
                    txtM22.Text = String.Format("{0,8:0.00000}",
reading.RotationMatrix.M22);
                    txtM23.Text = String.Format("{0,8:0.00000}",
reading.RotationMatrix.M23);
                    txtM31.Text = String.Format("{0,8:0.00000}",
reading.RotationMatrix.M31);
                    txtM32.Text = String.Format("{0,8:0.00000}",
reading.RotationMatrix.M32);
                    txtM33.Text = String.Format("{0,8:0.00000}",
reading.RotationMatrix.M33);
                });
```

```
            }

            public MainPage()
            {
                this.InitializeComponent();
                _sensor = OrientationSensor.GetDefault();

                // Establish the report interval for all scenarios
                uint minReportInterval = _sensor.MinimumReportInterval;
                uint reportInterval = minReportInterval > 16 ?
minReportInterval : 16;
                _sensor.ReportInterval = reportInterval;

                // Establish event handler
                _sensor.ReadingChanged += new
TypedEventHandler<OrientationSensor,
OrientationSensorReadingChangedEventArgs>(ReadingChanged);
            }
        }
    }
```

You'll need to rename the namespace in the previous snippet with the name you gave your project. For example, if you created a project named **OrientationSensorCS**, you'd replace `namespace App1` with `namespace OrientationSensorCS`.

- Open the file MainPage.xaml and replace the original contents with the following XML.

XML

```
        <Page
        x:Class="App1.MainPage"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="using:App1"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
        mc:Ignorable="d">

        <Grid x:Name="LayoutRoot" Background="Black">
            <TextBlock HorizontalAlignment="Left" Height="28"
Margin="4,4,0,0" TextWrapping="Wrap" Text="M11:" VerticalAlignment="Top"
Width="46"/>
            <TextBlock HorizontalAlignment="Left" Height="23"
Margin="4,36,0,0" TextWrapping="Wrap" Text="M12:" VerticalAlignment="Top"
Width="39"/>
            <TextBlock HorizontalAlignment="Left" Height="24"
Margin="4,72,0,0" TextWrapping="Wrap" Text="M13:" VerticalAlignment="Top"
Width="39"/>
            <TextBlock HorizontalAlignment="Left" Height="31"
Margin="4,118,0,0" TextWrapping="Wrap" Text="M21:" VerticalAlignment="Top"
```

```
Width="39"/>
            <TextBlock HorizontalAlignment="Left" Height="24"
Margin="4,160,0,0" TextWrapping="Wrap" Text="M22:" VerticalAlignment="Top"
Width="39"/>
            <TextBlock HorizontalAlignment="Left" Height="24"
Margin="8,201,0,0" TextWrapping="Wrap" Text="M23:" VerticalAlignment="Top"
Width="35"/>
            <TextBlock HorizontalAlignment="Left" Height="23"
Margin="4,234,0,0" TextWrapping="Wrap" Text="M31:" VerticalAlignment="Top"
Width="39"/>
            <TextBlock HorizontalAlignment="Left" Height="28"
Margin="4,274,0,0" TextWrapping="Wrap" Text="M32:" VerticalAlignment="Top"
Width="46"/>
            <TextBlock HorizontalAlignment="Left" Height="21"
Margin="4,322,0,0" TextWrapping="Wrap" Text="M33:" VerticalAlignment="Top"
Width="39"/>
            <TextBlock x:Name="txtM11" HorizontalAlignment="Left"
Height="19" Margin="43,4,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53"/>
            <TextBlock x:Name="txtM12" HorizontalAlignment="Left"
Height="23" Margin="43,36,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53"/>
            <TextBlock x:Name="txtM13" HorizontalAlignment="Left"
Height="15" Margin="43,72,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53"/>
            <TextBlock x:Name="txtM21" HorizontalAlignment="Left"
Height="20" Margin="43,114,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53"/>
            <TextBlock x:Name="txtM22" HorizontalAlignment="Left"
Height="19" Margin="43,156,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53"/>
            <TextBlock x:Name="txtM23" HorizontalAlignment="Left"
Height="16" Margin="43,197,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53"/>
            <TextBlock x:Name="txtM31" HorizontalAlignment="Left"
Height="17" Margin="43,230,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53"/>
            <TextBlock x:Name="txtM32" HorizontalAlignment="Left"
Height="19" Margin="43,270,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53"/>
            <TextBlock x:Name="txtM33" HorizontalAlignment="Left"
Height="21" Margin="43,322,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="53"/>
            <TextBlock HorizontalAlignment="Left" Height="15"
Margin="194,8,0,0" TextWrapping="Wrap" Text="Quaternion X:"
VerticalAlignment="Top" Width="81"/>
            <TextBlock HorizontalAlignment="Left" Height="23"
Margin="194,36,0,0" TextWrapping="Wrap" Text="Quaternion Y:"
VerticalAlignment="Top" Width="81"/>
            <TextBlock HorizontalAlignment="Left" Height="15"
Margin="194,72,0,0" TextWrapping="Wrap" Text="Quaternion Z:"
VerticalAlignment="Top" Width="81"/>
            <TextBlock x:Name="txtQuaternionX" HorizontalAlignment="Left"
Height="15" Margin="279,8,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="104"/>
```

```
            <TextBlock x:Name="txtQuaternionY" HorizontalAlignment="Left"
Height="12" Margin="275,36,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="108"/>
            <TextBlock x:Name="txtQuaternionZ" HorizontalAlignment="Left"
Height="19" Margin="275,68,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="89"/>
            <TextBlock HorizontalAlignment="Left" Height="21"
Margin="194,96,0,0" TextWrapping="Wrap" Text="Quaternion W:"
VerticalAlignment="Top" Width="81"/>
            <TextBlock x:Name="txtQuaternionW" HorizontalAlignment="Left"
Height="12" Margin="279,96,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="72"/>

        </Grid>
    </Page>
```

You'll need to replace the first part of the class name in the previous snippet with the namespace of your app. For example, if you created a project named **OrientationSensorCS**, you'd replace `x:Class="App1.MainPage"` with `x:Class="OrientationSensorCS.MainPage"`. You should also replace `xmlns:local="using:App1"` with `xmlns:local="using:OrientationSensorCS"`.

- Press F5 or select **Debug** > **Start Debugging** to build, deploy, and run the app.

Once the app is running, you can change the orientation by moving the device or using the emulator tools.

- Stop the app by returning to Visual Studio and pressing Shift+F5 or select **Debug** > **Stop Debugging** to stop the app.

Explanation

The previous example demonstrates how little code you'll need to write in order to integrate orientation-sensor input in your app.

The app establishes a connection with the default orientation sensor in the **MainPage** method.

C#

```
_sensor = OrientationSensor.GetDefault();
```

The app establishes the report interval within the **MainPage** method. This code retrieves the minimum interval supported by the device and compares it to a requested interval of 16 milliseconds (which approximates a 60-Hz refresh rate). If the minimum supported

interval is greater than the requested interval, the code sets the value to the minimum. Otherwise, it sets the value to the requested interval.

```
C#

uint minReportInterval = _sensor.MinimumReportInterval;
uint reportInterval = minReportInterval > 16 ? minReportInterval : 16;
_sensor.ReportInterval = reportInterval;
```

The new sensor data is captured in the **ReadingChanged** method. Each time the sensor driver receives new data from the sensor, it passes the values to your app using this event handler. The app registers this event handler on the following line.

```
C#

_sensor.ReadingChanged += new TypedEventHandler<OrientationSensor,
OrientationSensorReadingChangedEventArgs>(ReadingChanged);
```

These new values are written to the TextBlocks found in the project's XAML.

Create a SimpleOrientation app

This section is divided into two subsections. The first subsection will take you through the steps necessary to create a simple orientation application from scratch. The following subsection explains the app you have just created.

Instructions

- Create a new project, choosing a **Blank App (Universal Windows)** from the **Visual C#** project templates.
- Open your project's MainPage.xaml.cs file and replace the existing code with the following.

```
C#

    using System;
    using System.Collections.Generic;
    using System.IO;
    using System.Linq;
    using Windows.Foundation;
    using Windows.Foundation.Collections;
    using Windows.UI.Xaml;
    using Windows.UI.Xaml.Controls;
    using Windows.UI.Xaml.Controls.Primitives;
```

```
    using Windows.UI.Xaml.Data;
    using Windows.UI.Xaml.Input;
    using Windows.UI.Xaml.Media;
    using Windows.UI.Xaml.Navigation;

    using Windows.UI.Core;
    using Windows.Devices.Sensors;
    // The Blank Page item template is documented at
https://go.microsoft.com/fwlink/p/?linkid=234238

    namespace App1
    {
        /// <summary>
        /// An empty page that can be used on its own or navigated to within
a Frame.
        /// </summary>
        public sealed partial class MainPage : Page
        {
            // Sensor and dispatcher variables
            private SimpleOrientationSensor _simpleorientation;

            // This event handler writes the current sensor reading to
            // a text block on the app' s main page.

            private async void OrientationChanged(object sender,
SimpleOrientationSensorOrientationChangedEventArgs e)
            {
                await Dispatcher.RunAsync(CoreDispatcherPriority.Normal, ()
=>
                {
                    SimpleOrientation orientation = e.Orientation;
                    switch (orientation)
                    {
                        case SimpleOrientation.NotRotated:
                            txtOrientation.Text = "Not Rotated";
                            break;
                        case
SimpleOrientation.Rotated90DegreesCounterclockwise:
                            txtOrientation.Text = "Rotated 90 Degrees
Counterclockwise";
                            break;
                        case
SimpleOrientation.Rotated180DegreesCounterclockwise:
                            txtOrientation.Text = "Rotated 180 Degrees
Counterclockwise";
                            break;
                        case
SimpleOrientation.Rotated270DegreesCounterclockwise:
                            txtOrientation.Text = "Rotated 270 Degrees
Counterclockwise";
                            break;
                        case SimpleOrientation.Faceup:
                            txtOrientation.Text = "Faceup";
                            break;
                        case SimpleOrientation.Facedown:
```

```
                            txtOrientation.Text = "Facedown";
                            break;
                        default:
                            txtOrientation.Text = "Unknown orientation";
                            break;
                    }
                });
            }

            public MainPage()
            {
                this.InitializeComponent();
                _simpleorientation = SimpleOrientationSensor.GetDefault();

                // Assign an event handler for the sensor orientation-
changed event
                if (_simpleorientation != null)
                {
                    _simpleorientation.OrientationChanged += new
TypedEventHandler<SimpleOrientationSensor,
SimpleOrientationSensorOrientationChangedEventArgs>(OrientationChanged);
                }
            }
        }
    }
```

You'll need to rename the namespace in the previous snippet with the name you gave your project. For example, if you created a project named **SimpleOrientationCS**, you'd replace `namespace App1` with `namespace SimpleOrientationCS`.

- Open the file MainPage.xaml and replace the original contents with the following XML.

XML

```
    <Page
        x:Class="App1.MainPage"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:local="using:App1"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006"
        mc:Ignorable="d">

        <Grid x:Name="LayoutRoot" Background="#FF0C0C0C">
            <TextBlock HorizontalAlignment="Left" Height="24"
Margin="8,8,0,0" TextWrapping="Wrap" Text="Current Orientation:"
VerticalAlignment="Top" Width="101" Foreground="#FFF8F7F7"/>
            <TextBlock x:Name="txtOrientation" HorizontalAlignment="Left"
Height="24" Margin="118,8,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="175" Foreground="#FFFEFAFA"/>
```

```
        </Grid>
    </Page>
```

You'll need to replace the first part of the class name in the previous snippet with the namespace of your app. For example, if you created a project named **SimpleOrientationCS**, you'd replace `x:Class="App1.MainPage"` with `x:Class="SimpleOrientationCS.MainPage"`. You should also replace `xmlns:local="using:App1"` with `xmlns:local="using:SimpleOrientationCS"`.

- Press F5 or select **Debug** > **Start Debugging** to build, deploy, and run the app.

Once the app is running, you can change the orientation by moving the device or using the emulator tools.

- Stop the app by returning to Visual Studio and pressing Shift+F5 or select **Debug** > **Stop Debugging** to stop the app.

Explanation

The previous example demonstrates how little code you'll need to write in order to integrate simple-orientation sensor input in your app.

The app establishes a connection with the default sensor in the **MainPage** method.

C#

```
_simpleorientation = SimpleOrientationSensor.GetDefault();
```

The new sensor data is captured in the **OrientationChanged** method. Each time the sensor driver receives new data from the sensor, it passes the values to your app using this event handler. The app registers this event handler on the following line.

C#

```
_simpleorientation.OrientationChanged += new
TypedEventHandler<SimpleOrientationSensor,
SimpleOrientationSensorOrientationChangedEventArgs>(OrientationChanged);
```

These new values are written to a TextBlock found in the project's XAML.

C#

```
<TextBlock HorizontalAlignment="Left" Height="24" Margin="8,8,0,0"
TextWrapping="Wrap" Text="Current Orientation:" VerticalAlignment="Top"
Width="101" Foreground="#FFF8F7F7"/>
 <TextBlock x:Name="txtOrientation" HorizontalAlignment="Left" Height="24"
Margin="118,8,0,0" TextWrapping="Wrap" Text="TextBlock"
VerticalAlignment="Top" Width="175" Foreground="#FFFEFAFA"/>
```

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Bluetooth

Article • 05/06/2023

This section contains articles on how to integrate Bluetooth into Universal Windows Platform (UWP) apps.

> **ⓘ Important**
>
> You must declare the "bluetooth" capability in *Package.appxmanifest*.
>
> `<Capabilities> <DeviceCapability Name="bluetooth" /> </Capabilities>`

There are two different bluetooth technologies that you can choose to implement in your app.

Classic Bluetooth (RFCOMM)

Before Bluetooth LE, devices commonly used this protocol to communicate using Bluetooth. This protocol is simple and useful for device-to-device communication without the need of energy savings. For more information about this protocol, including code samples, see the Bluetooth RFCOMM topic.

Bluetooth Low-Energy (LE)

Bluetooth Low Energy (LE) is a specification that defines protocols for discovery and communication between devices that have an efficient energy usage requirement. For more information including code samples, see the Bluetooth Low Energy topic.

See Also

- Bluetooth developer FAQ

Bluetooth RFCOMM

Article • 05/06/2023

This topic provides an overview of Bluetooth RFCOMM in Universal Windows Platform (UWP) apps, along with example code on how to send or receive a file.

Important APIs

- Windows.Devices.Bluetooth
- Windows.Devices.Bluetooth.Rfcomm

> **ⓘ Important**
>
> You must declare the "bluetooth" capability in *Package.appxmanifest*.
>
> `<Capabilities> <DeviceCapability Name="bluetooth" /> </Capabilities>`

Overview

The APIs in the Windows.Devices.Bluetooth.Rfcomm namespace build on existing patterns for Windows.Devices, including enumeration and instantiation. Data reading and writing is designed to take advantage of established data stream patterns and objects in Windows.Storage.Streams. Service Discovery Protocol (SDP) attributes have a value and an expected type. However, some common devices have faulty implementations of SDP attributes where the value is not of the expected type. Additionally, many usages of RFCOMM do not require additional SDP attributes at all. For these reasons, this API offers access to the unparsed SDP data, from which developers can obtain the information they need.

The RFCOMM APIs use the concept of service identifiers. Although a service identifier is simply a 128-bit GUID, it is also commonly specified as either a 16- or 32-bit integer. The RFCOMM API offers a wrapper for service identifiers that allows them be specified and consumed as 128-bit GUIDs as well as 32-bit integers but does not offer 16-bit integers. This is not an issue for the API because languages will automatically upsize to a 32-bit integer and the identifier can still be correctly generated.

Apps can perform multi-step device operations in a background task so that they can run to completion even if the app is moved to the background and suspended. This allows for reliable device servicing such as changes to persistent settings or firmware, and content synchronization, without requiring the user to sit and watch a progress bar.

Use the DeviceServicingTrigger for device servicing and the DeviceUseTrigger for content synchronization. Note that these background tasks constrain the amount of time the app can run in the background, and are not intended to allow indefinite operation or infinite synchronization.

For a complete code sample that details RFCOMM operation, see the Bluetooth Rfcomm Chat Sample on Github.

Send a file as a client

When sending a file, the most basic scenario is to connect to a paired device based on a desired service. This involves the following steps:

- Use the **RfcommDeviceService.GetDeviceSelector*** functions to help generate an Advanced Query Syntax (AQS) query that can be used to enumerated paired device instances of the desired service.
- Pick an enumerated device, create an RfcommDeviceService, and read the SDP attributes as needed (using established data helpers to parse the attribute's data).
- Create a socket and use the RfcommDeviceService.ConnectionHostName and RfcommDeviceService.ConnectionServiceName properties to StreamSocket.ConnectAsync to the remote device service with the appropriate parameters.
- Follow established data stream patterns to read chunks of data from the file and send it on the socket's StreamSocket.OutputStream to the device.

C#

```
using System;
using System.Threading.Tasks;
using Windows.Devices.Bluetooth.Rfcomm;
using Windows.Networking.Sockets;
using Windows.Storage.Streams;
using Windows.Devices.Bluetooth;

Windows.Devices.Bluetooth.Rfcomm.RfcommDeviceService _service;
Windows.Networking.Sockets.StreamSocket _socket;

async void Initialize()
{
    // Enumerate devices with the object push service
    var services =
        await Windows.Devices.Enumeration.DeviceInformation.FindAllAsync(
            RfcommDeviceService.GetDeviceSelector(
                RfcommServiceId.ObexObjectPush));

    if (services.Count > 0)
    {
```

```
        // Initialize the target Bluetooth BR device
        var service = await RfcommDeviceService.FromIdAsync(services[0].Id);

        bool isCompatibleVersion = await IsCompatibleVersionAsync(service);

        // Check that the service meets this App's minimum requirement
        if (SupportsProtection(service) && isCompatibleVersion)
        {
            _service = service;

            // Create a socket and connect to the target
            _socket = new StreamSocket();
            await _socket.ConnectAsync(
                _service.ConnectionHostName,
                _service.ConnectionServiceName,
                SocketProtectionLevel
                    .BluetoothEncryptionAllowNullAuthentication);

            // The socket is connected. At this point the App can wait for
            // the user to take some action, for example, click a button to
send a
            // file to the device, which could invoke the Picker and then
            // send the picked file. The transfer itself would use the
            // Sockets API and not the Rfcomm API, and so is omitted here
for
            // brevity.
        }
    }
}

// This App requires a connection that is encrypted but does not care about
// whether it's authenticated.
bool SupportsProtection(RfcommDeviceService service)
{
    switch (service.ProtectionLevel)
    {
        case SocketProtectionLevel.PlainSocket:
            if ((service.MaxProtectionLevel == SocketProtectionLevel
                    .BluetoothEncryptionWithAuthentication)
                || (service.MaxProtectionLevel == SocketProtectionLevel
                    .BluetoothEncryptionAllowNullAuthentication))
            {
                // The connection can be upgraded when opening the socket so
the
                // App may offer UI here to notify the user that Windows may
                // prompt for a PIN exchange.
                return true;
            }
            else
            {
                // The connection cannot be upgraded so an App may offer UI
here
                // to explain why a connection won't be made.
                return false;
            }
```

```
        case SocketProtectionLevel.BluetoothEncryptionWithAuthentication:
            return true;
        case
SocketProtectionLevel.BluetoothEncryptionAllowNullAuthentication:
            return true;
    }
    return false;
}

// This App relies on CRC32 checking available in version 2.0 of the
service.
const uint SERVICE_VERSION_ATTRIBUTE_ID = 0x0300;
const byte SERVICE_VERSION_ATTRIBUTE_TYPE = 0x0A;   // UINT32
const uint MINIMUM_SERVICE_VERSION = 200;
async Task<bool> IsCompatibleVersionAsync(RfcommDeviceService service)
{
    var attributes = await service.GetSdpRawAttributesAsync(
        BluetoothCacheMode.Uncached);
    var attribute = attributes[SERVICE_VERSION_ATTRIBUTE_ID];
    var reader = DataReader.FromBuffer(attribute);

    // The first byte contains the attribute's type
    byte attributeType = reader.ReadByte();
    if (attributeType == SERVICE_VERSION_ATTRIBUTE_TYPE)
    {
        // The remainder is the data
        uint version = reader.ReadUInt32();
        return version >= MINIMUM_SERVICE_VERSION;
    }
    else return false;
}
```

Receive File as a Server

Another common RFCOMM App scenario is to host a service on the PC and expose it for other devices.

- Create a **RfcommServiceProvider** to advertise the desired service.
- Set the SDP attributes as needed (using **established data helpers** to generate the attribute's Data) and starts advertising the SDP records for other devices to retrieve.
- To connect to a client device, create a socket listener to start listening for incoming connection requests.
- When a connection is received, store the connected socket for later processing.
- Follow established data stream patterns to read chunks of data from the socket's InputStream and save it to a file.

In order to persist an RFCOMM service in the background, use the RfcommConnectionTrigger. The background task is triggered on connection to the service. The developer receives a handle to the socket in the background task. The background task is long running and persists for as long as the socket is in use.

C#

```
Windows.Devices.Bluetooth.Rfcomm.RfcommServiceProvider _provider;

async void Initialize()
{
    // Initialize the provider for the hosted RFCOMM service
    _provider =
        await
Windows.Devices.Bluetooth.Rfcomm.RfcommServiceProvider.CreateAsync(
            RfcommServiceId.ObexObjectPush);

    // Create a listener for this service and start listening
    StreamSocketListener listener = new StreamSocketListener();
    listener.ConnectionReceived += OnConnectionReceivedAsync;
    await listener.BindServiceNameAsync(
        _provider.ServiceId.AsString(),
        SocketProtectionLevel
            .BluetoothEncryptionAllowNullAuthentication);

    // Set the SDP attributes and start advertising
    InitializeServiceSdpAttributes(_provider);
    _provider.StartAdvertising(listener);
}

const uint SERVICE_VERSION_ATTRIBUTE_ID = 0x0300;
const byte SERVICE_VERSION_ATTRIBUTE_TYPE = 0x0A;   // UINT32
const uint SERVICE_VERSION = 200;

void InitializeServiceSdpAttributes(RfcommServiceProvider provider)
{
    Windows.Storage.Streams.DataWriter writer = new
Windows.Storage.Streams.DataWriter();

    // First write the attribute type
    writer.WriteByte(SERVICE_VERSION_ATTRIBUTE_TYPE);
    // Then write the data
    writer.WriteUInt32(MINIMUM_SERVICE_VERSION);

    IBuffer data = writer.DetachBuffer();
    provider.SdpRawAttributes.Add(SERVICE_VERSION_ATTRIBUTE_ID, data);
}

void OnConnectionReceivedAsync(
    StreamSocketListener listener,
    StreamSocketListenerConnectionReceivedEventArgs args)
{
    // Stop advertising/listening so that we're only serving one client
```

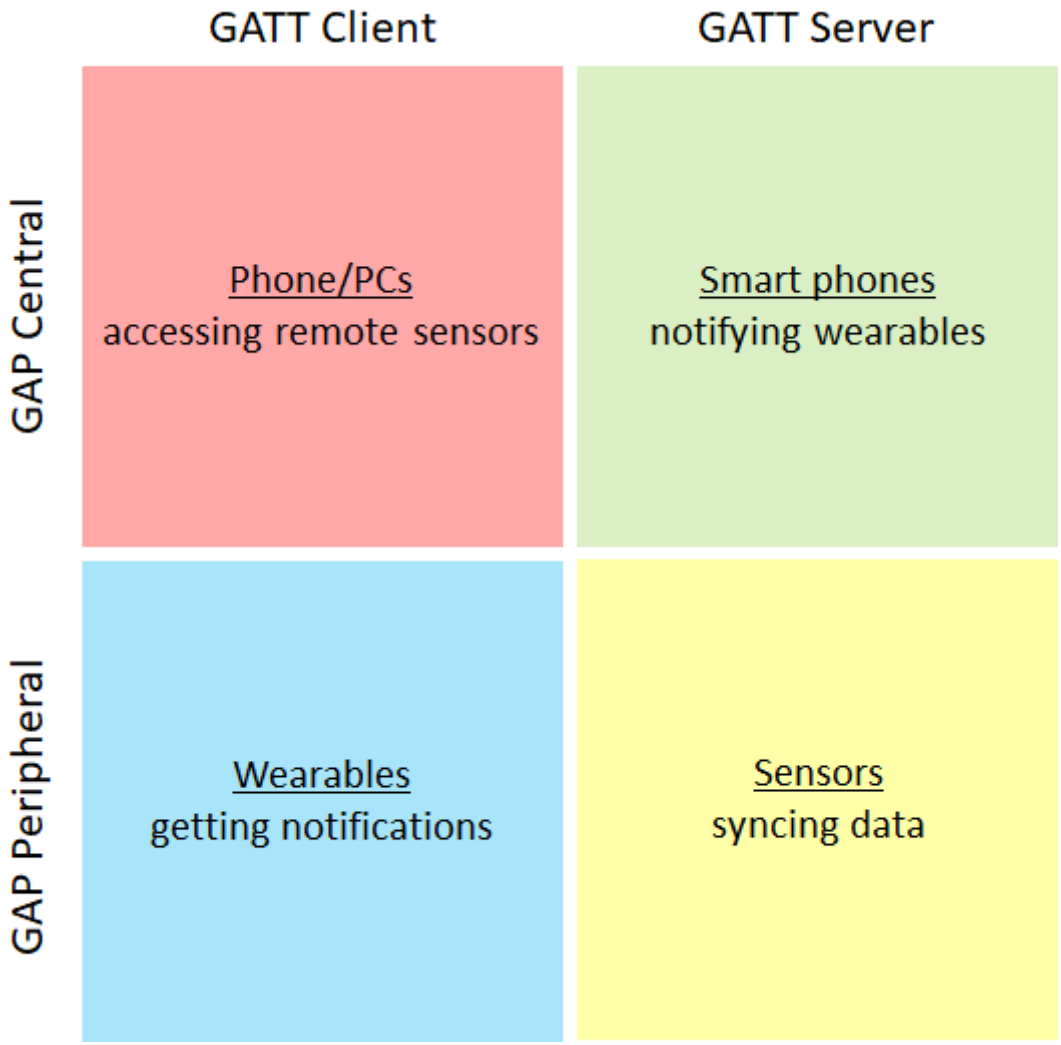
```
    _provider.StopAdvertising();
    listener.Dispose();
    _socket = args.Socket;

    // The client socket is connected. At this point the App can wait for
    // the user to take some action, for example, click a button to receive
a file
    // from the device, which could invoke the Picker and then save the
    // received file to the picked location. The transfer itself would use
    // the Sockets API and not the Rfcomm API, and so is omitted here for
    // brevity.
}
```

Bluetooth Low Energy in Universal Windows Platform apps

Article • 05/06/2023

This topic provides an overview of Bluetooth LE in Universal Windows Platform (UWP) apps (for more detail about Bluetooth LE, see the Bluetooth Core Specification version 4.0).

Bluetooth Low Energy (LE) is a specification that defines protocols for discovery and communication between power-efficient devices. Discovery of devices is done through the Generic Access Profile (GAP) protocol. After discovery, device-to-device communication is done through the Generic Attribute (GATT) protocol.



GATT and GAP roles were introduced in Windows 10 version 1703

GATT and GAP protocols can be implemented in your UWP app by using the following namespaces.

- Windows.Devices.Bluetooth.GenericAttributeProfile

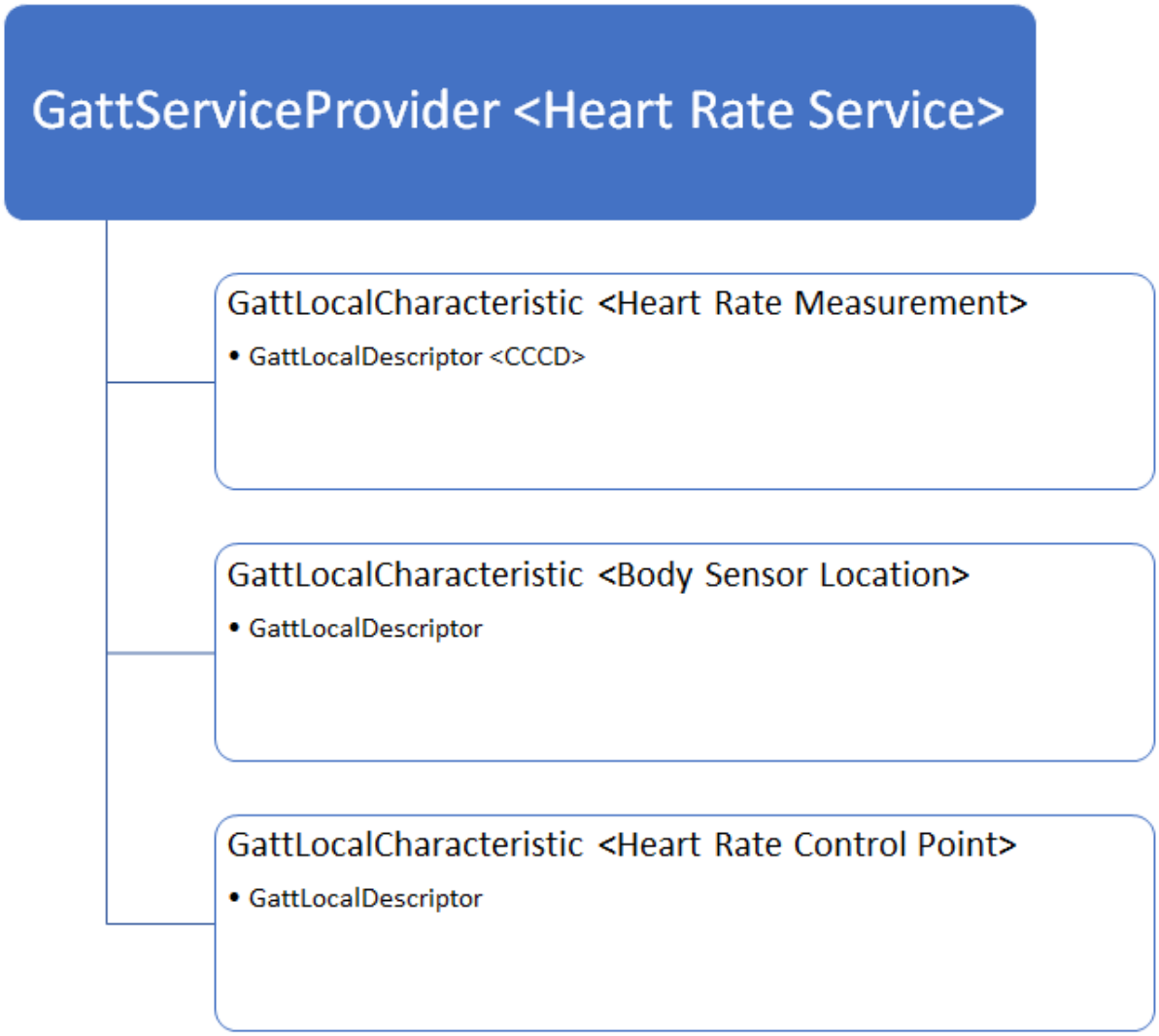
- Windows.Devices.Bluetooth.Advertisement

Central and Peripheral

The two primary roles of discovery are called Central and Peripheral. In general, Windows operates in Central mode and connects to various Peripheral devices.

Attributes

A cGeneric Attribute (GATT) Profile defines the structure of data and modes of operation by which two Bluetooth LE devices communicate. The attribute is the main building block of GATT. The main types of attributes are services, characteristics, and descriptors. These attributes perform differently between clients and servers, so it is more useful to discuss their interaction in the relevant sections.



The heart rate service is expressed in GATT Server API form

Client and Server

After a connection has been established, the device that contains the data (usually a small IoT sensor or wearable) is known as the Server. The device that uses that data to perform a function is known as the Client. For example, a Windows PC (Client) reads data from a heart rate monitor (Server) to track that a user is working out optimally. For more information, see the GATT Client and GATT Server topics.

Watchers and Publishers (Beacons)

In addition to the Central and Peripheral roles, there are Observer and Broadcaster roles. Broadcasters are commonly referred to as Beacons, they don't communicate over GATT because they use the limited space provided in the Advertisement packet for communication. Similarly, an Observer does not have to establish a connection to receive data, it scans for nearby advertisements. To configure Windows to observe nearby advertisements, use the BluetoothLEAdvertisementWatcher class. In order to broadcast beacon payloads, use the BluetoothLEAdvertisementPublisher class. For more information, see Bluetooth LE Advertisements.

See Also

- Windows.Devices.Bluetooth.GenericAttributeProfile
- Windows.Devices.Bluetooth.Advertisement
- Bluetooth Core Specification

Bluetooth GATT Client

Article • 05/06/2023

This article demonstrates how to use the Bluetooth Generic Attribute (GATT) Client APIs for Universal Windows Platform (UWP) apps.

> **ⓘ Important**
>
> You must declare the "bluetooth" capability in *Package.appxmanifest*.
>
> `<Capabilities> <DeviceCapability Name="bluetooth" /> </Capabilities>`

> **Important APIs**
>
> - Windows.Devices.Bluetooth
> - Windows.Devices.Bluetooth.GenericAttributeProfile

Overview

Developers can use the APIs in the Windows.Devices.Bluetooth.GenericAttributeProfile namespace to access Bluetooth LE devices. Bluetooth LE devices expose their functionality through a collection of:

- Services
- Characteristics
- Descriptors

Services define the functional contract of the LE device and contain a collection of characteristics that define the service. Those characteristics, in turn, contain descriptors that describe the characteristics. These 3 terms are generically known as the attributes of a device.

The Bluetooth LE GATT APIs expose objects and functions, rather than access to the raw transport. The GATT APIs also enable developers to work with Bluetooth LE devices with the ability to perform the following tasks:

- Perform attribute discovery
- Read and Write attribute values
- Register a callback for Characteristic ValueChanged event

To create a useful implementation a developer must have prior knowledge of the GATT services and characteristics the application intends to consume and to process the specific characteristic values such that the binary data provided by the API is transformed into useful data before being presented to the user. The Bluetooth GATT APIs expose only the basic primitives required to communicate with a Bluetooth LE device. To interpret the data, an application profile must be defined, either by a Bluetooth SIG standard profile, or a custom profile implemented by a device vendor. A profile creates a binding contract between the application and the device, as to what the exchanged data represents and how to interpret it.

For convenience the Bluetooth SIG maintains a list of public profiles available.

Examples

For a complete sample, see Bluetooth Low Energy sample.

Query for nearby devices

There are two main methods to query for nearby devices:

- DeviceWatcher in Windows.Devices.Enumeration
- BluetoothLEAdvertisementWatcher in Windows.Devices.Bluetooth.Advertisement

The 2nd method is discussed at length in the Advertisement documentation so it won't be discussed much here but the basic idea is to find the Bluetooth address of nearby devices that satisfy the particular Advertisement Filter. Once you have the address, you can call BluetoothLEDevice.FromBluetoothAddressAsync to get a reference to the device.

Now, back to the DeviceWatcher method. A Bluetooth LE device is just like any other device in Windows and can be queried using the Enumeration APIs. Use the DeviceWatcher class and pass a query string specifying the devices to look for:

C#

```
// Query for extra properties you want returned
string[] requestedProperties = { "System.Devices.Aep.DeviceAddress",
"System.Devices.Aep.IsConnected" };

DeviceWatcher deviceWatcher =
            DeviceInformation.CreateWatcher(

BluetoothLEDevice.GetDeviceSelectorFromPairingState(false),
                    requestedProperties,
                    DeviceInformationKind.AssociationEndpoint);
```

```
// Register event handlers before starting the watcher.
// Added, Updated and Removed are required to get all nearby devices
deviceWatcher.Added += DeviceWatcher_Added;
deviceWatcher.Updated += DeviceWatcher_Updated;
deviceWatcher.Removed += DeviceWatcher_Removed;

// EnumerationCompleted and Stopped are optional to implement.
deviceWatcher.EnumerationCompleted += DeviceWatcher_EnumerationCompleted;
deviceWatcher.Stopped += DeviceWatcher_Stopped;

// Start the watcher.
deviceWatcher.Start();
```

Once you've started the DeviceWatcher, you will receive DeviceInformation for each device that satisfies the query in the handler for the Added event for the devices in question. For a more detailed look at DeviceWatcher see the complete sample on Github.

Connecting to the device

Once a desired device is discovered, use the DeviceInformation.Id to get the Bluetooth LE Device object for the device in question:

C#

```
async void ConnectDevice(DeviceInformation deviceInfo)
{
    // Note: BluetoothLEDevice.FromIdAsync must be called from a UI thread
because it may prompt for consent.
    BluetoothLEDevice bluetoothLeDevice = await
BluetoothLEDevice.FromIdAsync(deviceInfo.Id);
    // ...
}
```

On the other hand, disposing of all references to a BluetoothLEDevice object for a device (and if no other app on the system has a reference to the device) will trigger an automatic disconnect after a small timeout period.

C#

```
bluetoothLeDevice.Dispose();
```

If the app needs to access the device again, simply re-creating the device object and accessing a characteristic (discussed in the next section) will trigger the OS to re-connect

when necessary. If the device is nearby, you'll get access to the device otherwise it will return w/ a DeviceUnreachable error.

> ⓘ Note
>
> Creating a BluetoothLEDevice object by calling this method alone doesn't (necessarily) initiate a connection. To initiate a connection, set GattSession.MaintainConnection to `true`, or call an uncached service discovery method on **BluetoothLEDevice**, or perform a read/write operation against the device.
>
> - If **GattSession.MaintainConnection** is set to true, then the system waits indefinitely for a connection, and it will connect when the device is available. There's nothing for your application to wait on, since **GattSession.MaintainConnection** is a property.
> - For service discovery and read/write operations in GATT, the system waits a finite but variable time. Anything from instantaneous to a matter of minutes. Factors include the traffic on the stack, and how queued up the request is. If there are no other pending request, and the remote device is unreachable, then the system will wait for seven (7) seconds before it times out. If there are other pending requests, then each of the requests in the queue can take seven (7) seconds to process, so the further yours is toward the back of the queue, the longer you'll wait.
>
> Currently, you can't cancel the connection process.

Enumerating supported services and characteristics

Now that you have a BluetoothLEDevice object, the next step is to discover what data the device exposes. The first step to do this is to query for services:

C#

```
GattDeviceServicesResult result = await
bluetoothLeDevice.GetGattServicesAsync();

if (result.Status == GattCommunicationStatus.Success)
{
    var services = result.Services;
    // ...
}
```

Once the service of interest has been identified, the next step is to query for characteristics.

```
C#

GattCharacteristicsResult result = await service.GetCharacteristicsAsync();

if (result.Status == GattCommunicationStatus.Success)
{
    var characteristics = result.Characteristics;
    // ...
}
```

The OS returns a ReadOnly list of GattCharacteristic objects that you can then perform operations on.

Perform Read/Write operations on a characteristic

The characteristic is the fundamental unit of GATT based communication. It contains a value that represents a distinct piece of data on the device. For example, the battery level characteristic has a value that represents the battery level of the device.

Read the characteristic properties to determine what operations are supported:

```
C#

GattCharacteristicProperties properties =
characteristic.CharacteristicProperties

if(properties.HasFlag(GattCharacteristicProperties.Read))
{
    // This characteristic supports reading from it.
}
if(properties.HasFlag(GattCharacteristicProperties.Write))
{
    // This characteristic supports writing to it.
}
if(properties.HasFlag(GattCharacteristicProperties.Notify))
{
    // This characteristic supports subscribing to notifications.
}
```

If read is supported, you can read the value:

```
C#

GattReadResult result = await selectedCharacteristic.ReadValueAsync();
if (result.Status == GattCommunicationStatus.Success)
```

```
{
    var reader = DataReader.FromBuffer(result.Value);
    byte[] input = new byte[reader.UnconsumedBufferLength];
    reader.ReadBytes(input);
    // Utilize the data as needed
}
```

Writing to a characteristic follows a similar pattern:

C#

```
var writer = new DataWriter();
// WriteByte used for simplicity. Other common functions - WriteInt16 and
WriteSingle
writer.WriteByte(0x01);

GattCommunicationStatus result = await
selectedCharacteristic.WriteValueAsync(writer.DetachBuffer());
if (result == GattCommunicationStatus.Success)
{
    // Successfully wrote to device
}
```

Tip

DataReader and DataWriter are indispensable when working with the raw buffers you get from many of the Bluetooth APIs.

Subscribing for notifications

Make sure the characteristic supports either Indicate or Notify (check the characteristic properties to make sure).

Indicate is considered more reliable because each value changed event is coupled with an acknowledgement from the client device. Notify is more prevalent because most GATT transactions would rather conserve power rather than be extremely reliable. In any case, all of that is handled at the controller layer so the app does not get involved. We'll collectively refer to them as simply "notifications".

There are two things to take care of before getting notifications:

- Write to Client Characteristic Configuration Descriptor (CCCD)
- Handle the Characteristic.ValueChanged event

Writing to the CCCD tells the Server device that this client wants to know each time that particular characteristic value changes. To do this:

C#

```
GattCommunicationStatus status = await
selectedCharacteristic.WriteClientCharacteristicConfigurationDescriptorAsync
(

GattClientCharacteristicConfigurationDescriptorValue.Notify);
if(status == GattCommunicationStatus.Success)
{
    // Server has been informed of clients interest.
}
```

Now, the GattCharacteristic's ValueChanged event will get called each time the value gets changed on the remote device. All that's left is to implement the handler:

C#

```
characteristic.ValueChanged += Characteristic_ValueChanged;

...

void Characteristic_ValueChanged(GattCharacteristic sender,
                                 GattValueChangedEventArgs args)
{
    // An Indicate or Notify reported that the value has changed.
    var reader = DataReader.FromBuffer(args.CharacteristicValue)
    // Parse the data however required.
}
```

Feedback

Bluetooth GATT Server

Article • 04/16/2024

This topic demonstrates how to use the Bluetooth Generic Attribute (GATT) Server APIs for Universal Windows Platform (UWP) apps.

> **ⓘ Important**
>
> You must declare the "bluetooth" capability in *Package.appxmanifest*.
>
> `<Capabilities> <DeviceCapability Name="bluetooth" /> </Capabilities>`

Important APIs

- Windows.Devices.Bluetooth
- Windows.Devices.Bluetooth.GenericAttributeProfile

Overview

Windows usually operates in the client role. Nevertheless, many scenarios arise which require Windows to act as a Bluetooth LE GATT Server as well. Almost all the scenarios for IoT devices, along with most cross-platform BLE communication will require Windows to be a GATT Server. Additionally, sending notifications to nearby wearable devices has become a popular scenario that requires this technology as well.

Server operations will revolve around the Service Provider and the GattLocalCharacteristic. These two classes will provide the functionality needed to declare, implement and expose a hierarchy of data to a remote device.

Define the supported services

Your app may declare one or more services that will be published by Windows. Each service is uniquely identified by a UUID.

Attributes and UUIDs

Each service, characteristic and descriptor is defined by it's own unique 128-bit UUID.

> The Windows APIs all use the term GUID, but the Bluetooth standard defines these as UUIDs. For our purposes, these two terms are interchangeable so we'll continue

> to use the term UUID.

If the attribute is standard and defined by the Bluetooth SIG-defined, it will also have a corresponding 16-bit short ID (for example, Battery Level UUID is 0000**2A19**-0000-1000-8000-00805F9B34FB and the short ID is 0x2A19). These standard UUIDs can be seen in GattServiceUuids and GattCharacteristicUuids.

If your app is implementing it's own custom service, a custom UUID will have to be generated. This is easily done in Visual Studio through Tools -> CreateGuid (use option 5 to get it in the "xxxxxxxx-xxxx-...xxxx" format). This uuid can now be used to declare new local services, characteristics or descriptors.

Restricted Services

The following Services are reserved by the system and cannot be published at this time:

1. Device Information Service (DIS)
2. Generic Attribute Profile Service (GATT)
3. Generic Access Profile Service (GAP)
4. Scan Parameters Service (SCP)

> Attempting to create a blocked service will result in BluetoothError.DisabledByPolicy being returned from the call to CreateAsync.

Generated Attributes

The following descriptors are autogenerated by the system, based on the GattLocalCharacteristicParameters provided during creation of the characteristic:

1. Client Characteristic Configuration (if the characteristic is marked as indicatable or notifiable).
2. Characteristic User Description (if UserDescription property is set). See GattLocalCharacteristicParameters.UserDescription property for more info.
3. Characteristic Format (one descriptor for each presentation format specified). See GattLocalCharacteristicParameters.PresentationFormats property for more info.
4. Characteristic Aggregate Format (if more than one presentation format is specified). GattLocalCharacteristicParameters.See PresentationFormats property for more info.
5. Characteristic Extended Properties (if the characteristic is marked with the extended properties bit).

> The value of the Extended Properties descriptor is determined via the ReliableWrites and WritableAuxiliaries characteristic properties.

> Attempting to create a reserved descriptor will result in an exception.

> Note that broadcast is not supported at this time. Specifying the Broadcast GattCharacteristicProperty will result in an exception.

Build up the hierarchy of services and characteristics

The GattServiceProvider is used to create and advertise the root primary service definition. Each service requires it's own ServiceProvider object that takes in a GUID:

C#

```
GattServiceProviderResult result = await
GattServiceProvider.CreateAsync(uuid);

if (result.Error == BluetoothError.Success)
{
    serviceProvider = result.ServiceProvider;
    //
}
```

> Primary services are the top level of the GATT tree. Primary services contain characteristics as well as other services (called 'Included' or secondary services).

Now, populate the service with the required characteristics and descriptors:

C#

```
GattLocalCharacteristicResult characteristicResult = await
serviceProvider.Service.CreateCharacteristicAsync(uuid1, ReadParameters);
if (characteristicResult.Error != BluetoothError.Success)
{
    // An error occurred.
    return;
}
_readCharacteristic = characteristicResult.Characteristic;
_readCharacteristic.ReadRequested += ReadCharacteristic_ReadRequested;

characteristicResult = await
serviceProvider.Service.CreateCharacteristicAsync(uuid2, WriteParameters);
if (characteristicResult.Error != BluetoothError.Success)
{
    // An error occurred.
```

```
    return;
}
_writeCharacteristic = characteristicResult.Characteristic;
_writeCharacteristic.WriteRequested += WriteCharacteristic_WriteRequested;

characteristicResult = await
serviceProvider.Service.CreateCharacteristicAsync(uuid3, NotifyParameters);
if (characteristicResult.Error != BluetoothError.Success)
{
    // An error occurred.
    return;
}
_notifyCharacteristic = characteristicResult.Characteristic;
_notifyCharacteristic.SubscribedClientsChanged += SubscribedClientsChanged;
```

As shown above, this is also a good place to declare event handlers for the operations each characteristic supports. To respond to requests correctly, an app must defined and set an event handler for each request type the attribute supports. Failing to register a handler will result in the request being completed immediately with *UnlikelyError* by the system.

Constant characteristics

Sometimes, there are characteristic values that will not change during the course of the app's lifetime. In that case, it is advisable to declare a constant characteristic to prevent unnecessary app activation:

C#

```
byte[] value = new byte[] {0x21};
var constantParameters = new GattLocalCharacteristicParameters
{
    CharacteristicProperties = (GattCharacteristicProperties.Read),
    StaticValue = value.AsBuffer(),
    ReadProtectionLevel = GattProtectionLevel.Plain,
};

var characteristicResult = await
serviceProvider.Service.CreateCharacteristicAsync(uuid4,
constantParameters);
if (characteristicResult.Error != BluetoothError.Success)
{
    // An error occurred.
    return;
}
```

Publish the service

Once the service has been fully defined, the next step is to publish support for the service. This informs the OS that the service should be returned when remote devices perform a service discovery. You will have to set two properties - IsDiscoverable and IsConnectable:

```
C#

GattServiceProviderAdvertisingParameters advParameters = new
GattServiceProviderAdvertisingParameters
{
    IsDiscoverable = true,
    IsConnectable = true
};
serviceProvider.StartAdvertising(advParameters);
```

- **IsDiscoverable**: Advertises the friendly name to remote devices in the advertisement, making the device discoverable.
- **IsConnectable**: Advertises a connectable advertisement for use in peripheral role.

> When a service is both Discoverable and Connectable, the system will add the Service Uuid to the advertisement packet. There are only 31 bytes in the Advertisement packet and a 128-bit UUID takes up 16 of them!

> Note that when a service is published in the foreground, an application must call StopAdvertising when the application suspends.

Respond to Read and Write requests

As we saw above while declaring the required characteristics, GattLocalCharacteristics have 3 types of events - ReadRequested, WriteRequested and SubscribedClientsChanged.

Read

When a remote device tries to read a value from a characteristic (and it's not a constant value), the ReadRequested event is called. The characteristic the read was called on as well as args (containing information about the remote device) is passed to the delegate:

```
C#

characteristic.ReadRequested += Characteristic_ReadRequested;
// ...

async void ReadCharacteristic_ReadRequested(GattLocalCharacteristic sender,
```

```
GattReadRequestedEventArgs args)
{
    var deferral = args.GetDeferral();

    // Our familiar friend - DataWriter.
    var writer = new DataWriter();
    // populate writer w/ some data.
    // ...

    var request = await args.GetRequestAsync();
    request.RespondWithValue(writer.DetachBuffer());

    deferral.Complete();
}
```

Write

When a remote device tries to write a value to a characteristic, the WriteRequested event is called with details about the remote device, which characteristic to write to and the value itself:

C#

```
characteristic.ReadRequested += Characteristic_ReadRequested;
// ...

async void WriteCharacteristic_WriteRequested(GattLocalCharacteristic
sender, GattWriteRequestedEventArgs args)
{
    var deferral = args.GetDeferral();

    var request = await args.GetRequestAsync();
    var reader = DataReader.FromBuffer(request.Value);
    // Parse data as necessary.

    if (request.Option == GattWriteOption.WriteWithResponse)
    {
        request.Respond();
    }

    deferral.Complete();
}
```

There are 2 types of Writes - with and without response. Use GattWriteOption (a property on the GattWriteRequest object) to figure out which type of write the remote device is performing.

Send notifications to subscribed clients

The most frequent of the GATT Server operations, notifications perform the critical function of pushing data to the remote devices. Sometimes, you'll want to notify all subscribed clients but other times you may want to pick which devices to send the new value to:

C#

```
async void NotifyValue()
{
    var writer = new DataWriter();
    // Populate writer with data
    // ...

    await notifyCharacteristic.NotifyValueAsync(writer.DetachBuffer());
}
```

When a new device subscribes for notifications, the SubscribedClientsChanged event gets called:

C#

```
characteristic.SubscribedClientsChanged += SubscribedClientsChanged;
// ...

void _notifyCharacteristic_SubscribedClientsChanged(GattLocalCharacteristic
sender, object args)
{
    List<GattSubscribedClient> clients = sender.SubscribedClients;
    // Diff the new list of clients from a previously saved one
    // to get which device has subscribed for notifications.

    // You can also just validate that the list of clients is expected for
this app.
}
```

> **Note**
>
> Your application can get the maximum notification size for a particular client with the **MaxNotificationSize** property. Any data larger than the maximum size will be truncated by the system.

When you handle the **GattLocalCharacteristic.SubscribedClientsChanged** event, you can use the process described below to determine full information about the currently subscribed client devices:

- The *args* of the **SubscribedClientsChanged** event is a **GattLocalCharacteristic** object.
- Access that object's **GattLocalCharacteristic.SubscribedClients** property, which is a collection of **GattSubscribedClient** objects.
- Iterate through that collection. For each element, do the following:
  - Access the **GattSubscribedClient.Session** property, which is a **GattSession** object.
  - Access the **GattSession.DeviceId** property, which is a **BluetoothDeviceId** object.
  - Access the **BluetoothDeviceId.Id** property, which is the device ID string.
  - Pass the device ID string to **BluetoothLEDevice.FromIdAsync** to retrieve a **BluetoothLEDevice** object. You can get full information about the device from that object.

Feedback

Bluetooth LE Advertisements

Article • 05/06/2023

This topic provides an overview of Bluetooth Low Energy (LE) Advertisement beacons for Universal Windows Platform (UWP) apps.

> **ⓘ Important**
>
> You must declare the "bluetooth" capability in *Package.appxmanifest* to use this feature.
>
> `<Capabilities> <DeviceCapability Name="bluetooth" /> </Capabilities>`

Important APIs

- **Windows.Devices.Bluetooth.Advertisement**

Supported features

There are two main features supported by the LE Advertisement APIs:

- Advertisement Watcher: listen for nearby beacons and filter them out based on payload or proximity.
- Advertisement Publisher: define a payload for Windows to advertise on a developers behalf.

Example

For a fully functional example of Bluetooth LE Advertisements, see the Bluetooth Advertisement Sample on Github.

Basic Setup

To use basic Bluetooth LE functionality in a Universal Windows Platform app, you must check the Bluetooth capability in the Package.appxmanifest.

1. Open Package.appxmanifest
2. Go to the Capabilities tab
3. Find Bluetooth in the list on the left and check the box next to it.

Publishing Advertisements

Bluetooth LE Advertisements allow your device to constantly beacon out a specific payload, called an advertisement. This advertisement can be seen by any nearby Bluetooth LE capable device, if they are set up to listen for this specific advertisement.

> **Note**
>
> For user privacy, the lifespan of your advertisement is tied to that of your app. You can create a BluetoothLEAdvertisementPublisher and call Start in a background task for advertisement in the background. For more information about background tasks, see Launching, resuming, and background tasks.

There are many different ways to add data to an Advertisement. This example shows a common way to create a company-specific advertisement.

First, create the advertisement publisher that controls whether or not the device is beaconing out a specific advertisement.

C#

```
BluetoothLEAdvertisementPublisher publisher = new
BluetoothLEAdvertisementPublisher();
```

Second, create a custom data section. This example uses an unassigned **CompanyId** value *0xFFFE* and adds the text *Hello World* to the advertisement.

C#

```
// Add custom data to the advertisement
var manufacturerData = new BluetoothLEManufacturerData();
manufacturerData.CompanyId = 0xFFFE;

var writer = new DataWriter();
writer.WriteString("Hello World");

// Make sure that the buffer length can fit within an advertisement payload
(~20 bytes).
// Otherwise you will get an exception.
manufacturerData.Data = writer.DetachBuffer();

// Add the manufacturer data to the advertisement publisher:
publisher.Advertisement.ManufacturerData.Add(manufacturerData);
```

Now that the publisher has been created and setup, you can call **Start** to begin advertising.

```
C#

publisher.Start();
```

Watching for Advertisements

The following code demonstrates how to create a Bluetooth LE Advertisement watcher, set a callback, and start watching for all LE advertisements.

```
C#

BluetoothLEAdvertisementWatcher watcher = new
BluetoothLEAdvertisementWatcher();
watcher.Received += OnAdvertisementReceived;
watcher.Start();
```

```
C#

private async void OnAdvertisementReceived(BluetoothLEAdvertisementWatcher
watcher, BluetoothLEAdvertisementReceivedEventArgs eventArgs)
{
    // Do whatever you want with the advertisement
}
```

Active Scanning

To receive scan response advertisements as well, set the following after creating the watcher. Note that this will cause greater power drain and is not available while in background modes.

```
C#

watcher.ScanningMode = BluetoothLEScanningMode.Active;
```

Watching for a Specific Advertisement Pattern

Sometimes you want to listen for a specific advertisement. In this case, listen for an advertisement containing a payload with a made up company (identified as 0xFFFE) and containing the string *Hello World* in the advertisement. This can be paired with the Basic

Publishing example to have one Windows machine advertising and another watching. Be sure to set this advertisement filter before you start the watcher!

C#

```
var manufacturerData = new BluetoothLEManufacturerData();
manufacturerData.CompanyId = 0xFFFE;

// Make sure that the buffer length can fit within an advertisement payload
(~20 bytes).
// Otherwise you will get an exception.
var writer = new DataWriter();
writer.WriteString("Hello World");
manufacturerData.Data = writer.DetachBuffer();

watcher.AdvertisementFilter.Advertisement.ManufacturerData.Add(manufacturerD
ata);
```

Watching for a Nearby Advertisement

Sometimes you only want to trigger your watcher when the device advertising has come in range. You can define your own range, just note that values will be clipped to between 0 and -128.

C#

```
// Set the in-range threshold to -70dBm. This means advertisements with RSSI
>= -70dBm
// will start to be considered "in-range" (callbacks will start in this
range).
watcher.SignalStrengthFilter.InRangeThresholdInDBm = -70;

// Set the out-of-range threshold to -75dBm (give some buffer). Used in
conjunction
// with OutOfRangeTimeout to determine when an advertisement is no longer
// considered "in-range".
watcher.SignalStrengthFilter.OutOfRangeThresholdInDBm = -75;

// Set the out-of-range timeout to be 2 seconds. Used in conjunction with
// OutOfRangeThresholdInDBm to determine when an advertisement is no longer
// considered "in-range"
watcher.SignalStrengthFilter.OutOfRangeTimeout =
TimeSpan.FromMilliseconds(2000);
```

Gauging Distance

When your Bluetooth LE Watcher's callback is triggered, the eventArgs include an RSSI value telling you the received signal strength (how strong the Bluetooth signal is).

C#

```
private async void OnAdvertisementReceived(BluetoothLEAdvertisementWatcher
watcher, BluetoothLEAdvertisementReceivedEventArgs eventArgs)
{
   // The received signal strength indicator (RSSI)
   Int16 rssi = eventArgs.RawSignalStrengthInDBm;
}
```

This can be roughly translated into distance, but should not be used to measure true distances as each individual radio is different. Different environmental factors can make distance difficult to gauge (such as walls, cases around the radio, or even air humidity).

An alternative to judging pure distance is to define "buckets". Radios tend to report 0 to -50 DBm when they are very close, -50 to -90 when they are a medium distance away, and below -90 when they are far away. Trial and error is best to determine what you want these buckets to be for your app.

Feedback

Bluetooth Developer FAQ

FAQ

This article contains answers to commonly asked UWP Bluetooth API questions.

What APIs do I use? Bluetooth Classic (RFCOMM) or Bluetooth Low Energy (GATT)?

There are various discussions online around this general topic so let's keep this answer squarely on the difference with respect to Windows. Here are some general guidelines:

Bluetooth LE (Windows.Devices.Bluetooth.GenericAttributeProfile)

Use the GATT APIs when you are communicating with a device that supports Bluetooth Low Energy. If your use case is infrequent, low bandwidth, or requires low power, Bluetooth Low Energy is the answer. The main namespace that includes this functionality is Windows.Devices.Bluetooth.GenericAttributeProfile.

When not to use Bluetooth LE

- High bandwidth, high frequency scenarios. If you need to constantly keep sync with large amounts of data, consider using Bluetooth classic or maybe even WiFi.

Bluetooth Classic (Windows.Devices.Bluetooth.Rfcomm)

The RFCOMM APIs give developers a socket to perform bidirectional serial port-style communication. Once you've got a socket, the methods for writing and reading are fairly standard. An implementation of this is presented in the Rfcomm Chat sample.

When not to use Bluetooth Rfcomm

- Notifications. The Bluetooth GATT protocol has a specific command for this and will result in significantly less power draw and faster response times.
- Checking for proximity or presence detection. Better to use the Advertisement APIs and connect over Bluetooth LE.

Why does my Bluetooth LE Device stop responding after a disconnect?

The most common reason this occurs is because the remote device has lost pairing information. A large number of older Bluetooth devices don't require authentication. To protect the user, all pairing transactions performed from the Settings application will require authentication, and some devices were not designed with this in mind.

Starting with Windows 10 release 1511, developers have control over the pairing handshake. The Device Enumeration and Pairing Sample details the various aspects of associating new devices.

In this example, we initiate pairing with a device using no encryption. Note, this will only work if the remote device does not require encryption or authentication to function.

C#

```
// Get ceremony type and protection level selections
// You must select at least ConfirmOnly or the pairing attempt will fail
    DevicePairingKinds ceremonySelected = DevicePairingKinds.ConfirmOnly;

//  Workaround remote devices losing pairing information
    DevicePairingProtectionLevel protectionLevel =
DevicePairingProtectionLevel.None

    DeviceInformationCustomPairing customPairing =
deviceInfoDisp.DeviceInformation.Pairing.Custom;

// Declare an event handler - you don't need to do much in
PairingRequestedHandler since the ceremony is "None"
    customPairing.PairingRequested += PairingRequestedHandler;
    DevicePairingResult result = await
customPairing.PairAsync(ceremonySelected, protectionLevel);
```

Do I have to pair Bluetooth devices before using them?

You don't have to pair devices before using them if leveraging Bluetooth RFCOMM (classic). Starting with Windows 10 release 1607, you can simply query for nearby devices and connect to them. The updated RFCOMM Chat Sample shows this functionality.

(14393 and below) This feature is not available for Bluetooth Low Energy (GATT Client), so you will still have to pair either through the Settings page or using the Windows.Devices.Enumeration APIs in order to access these devices.

(15030 and above) Pairing Bluetooth devices is no longer needed. Use the new Async APIs like GetGattServicesAsync and GetCharacteristicsAsync in order to query the current state of the remote device. See the Client docs for more details.

When should I pair with a device before communicating with it?

Generally, if you require a trusted, long-term bond with a device, pair with it by either directing the user to the settings page or using the Device Enumeration and Pairing APIs. If you simply need to read information from the device that is exposed publicly (a temperature sensor or beacon), then connect or listen for advertisements without making any effort to pair with the device. This will prevent interoperability problems in the long run, because a large number of devices do not support pairing.

Do all Windows devices support Peripheral Role?

No. This is a hardware-dependent feature, but a method is provided, BluetoothAdapter.IsPeripheralRoleSupported, to query whether it is supported or not. Currently supported devices include Windows Phone on 8992+ and RPi3 (Windows IoT).

Can I access these APIs from Win32?

Yes, all these APIs should work. This blog details the way to call Windows APIs from Desktop applications.

Is this functionality supposed to exist on a specific SKU?

Bluetooth LE: Yes, all functionality is in OneCore and should be available on most recent devices w/ a functioning Bluetooth LE stack.

> Caveat: Peripheral Role is hardware-dependent and some Windows Server Editions don't support Bluetooth.

Bluetooth BR/EDR (Classic): Some variations exist but mostly, they have very similar profile level support. See the docs on RFCOMM and these supported profile documents for PC and Phone

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

3D printing

Article • 05/06/2023

This section describes how to use the Windows.Graphics.Printing3D namespace to add 3D printing functionality to your Universal Windows Platform (UWP) app.

For more information on 3D printing with Windows, including resources for hardware partners, community discussion forums, and general information on 3D print capabilities, see 3D printing with Windows 10.

Topic	Description
3D print from your app	Learn how to add 3D printing functionality to your UWP app. This topic covers how to launch the 3D print dialog after ensuring your 3D model is printable and in the correct format.
Generate a 3MF package	Describes the structure of the 3D Manufacturing Format file type and how it can be created and manipulated with the Windows.Graphics.Printing3D APIs.

Related topics

- Windows.Graphics.Printing3D namespace
- 3D printing sample
- 3D printing from Unity sample

3D printing from your Universal Windows Platform app

Article • 05/06/2023

Learn how to add 3D printing functionality to your Universal Windows Platform (UWP) app.

This topic covers how to load 3D geometry data into your app and launch the 3D print dialog after ensuring your 3D model is printable and in the correct format. For a working example of these procedures, see the 3D printing UWP sample.

Important APIs

- Windows.Graphics.Printing3D

Setup

Add the Windows.Graphics.Printing3D namespace to your application class that requires 3D print functionality.

C#

```
using Windows.Graphics.Printing3D;
```

The following namespaces will also be used in this guide.

C#

```
using System;
using Windows.Storage;
using Windows.Storage.Pickers;
using Windows.Storage.Streams;
using Windows.System;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
```

Next, give your class helpful member fields.

- Declare a Print3DTask object to represent the printing task to be passed to the print driver.
- Declare a StorageFile object to hold the original 3D data file that will be loaded into the app.

- Declare a **Printing3D3MFPackage** object to represent a print-ready 3D model with all necessary metadata.

C#

```
private Print3DTask printTask;
private StorageFile file;
private Printing3D3MFPackage package = new Printing3D3MFPackage();
```

Create a simple UI

This sample uses a *Load* button to bring a file into program memory, a *Fix* button to make any needed modifications to the file, and a *Print* button to initiate the print job. The following code creates these buttons (with their on-click event handlers) in the corresponding XAML file of your .cs class.

XML

```
<StackPanel Orientation="Vertical" VerticalAlignment="Center">
    <Button x:Name="loadbutton" Content="Load Model from File"
HorizontalAlignment="Center" Margin="5,5,5,5" VerticalAlignment="Center"
Click="OnLoadClick"/>
    <Button x:Name="fixbutton" Content="Fix Model"
HorizontalAlignment="Center" Margin="5,5,5,5" VerticalAlignment="Center"
Click="OnFixClick"/>
    <Button x:Name="printbutton" Content="Print"
HorizontalAlignment="center" Margin="5,5,5,5" VerticalAlignment="Center"
Click="OnPrintClick"/>
```

The sample also includes a **TextBlock** for UI feedback.

XML

```
    <TextBlock x:Name="OutputTextBlock" TextAlignment="Center"></TextBlock>
</StackPanel>
```

Get the 3D data

The method by which your app acquires 3D geometry data can vary. Your app may retrieve data from a 3D scan, download model data from a web resource, or generate a 3D mesh programmatically using mathematical formulas or user input. Here, we show how to load a 3D data file (of any of several common file types) into program memory

from device storage. The 3D Builder model library provides a variety of models for you to download.

In the `OnLoadClick` method, the **FileOpenPicker** class loads a single file into app memory.

The following code shows how to load a single file into app memory using the **FileOpenPicker** class in the `OnLoadClick` method.

C#

```
private async void OnLoadClick(object sender, RoutedEventArgs e) {

    FileOpenPicker openPicker = new FileOpenPicker();

    // allow common 3D data file types
    openPicker.FileTypeFilter.Add(".3mf");
    openPicker.FileTypeFilter.Add(".stl");
    openPicker.FileTypeFilter.Add(".ply");
    openPicker.FileTypeFilter.Add(".obj");

    // pick a file and assign it to this class' 'file' member
    file = await openPicker.PickSingleFileAsync();
    if (file == null) {
        return;
    }
```

Use 3D Builder to convert to 3D Manufacturing Format (.3mf)

3D geometry data can come in many different formats, and not all are efficient for 3D printing. Windows uses the 3D Manufacturing Format (.3mf) file type for all 3D printing tasks.

See the 3MF Specification to learn more about 3MF and the supported features for producers and consumers of 3D product. To learn how to utilize these features with Windows APIs, see the Generate a 3MF package tutorial.

> **Note**
>
> The 3D Builder app can open files of most popular 3D formats and save them as .3mf files. It also provides tools to edit your models, add color data, and perform other print-specific operations.

In this example, where the file type could vary, you could open the 3D Builder app and prompt the user to save the imported data as a .3mf file and then reload it.

C#

```
    // if user loaded a non-3mf file type
    if (file.FileType != ".3mf") {

        // elect 3D Builder as the application to launch
        LauncherOptions options = new LauncherOptions();
        options.TargetApplicationPackageFamilyName =
"Microsoft.3DBuilder_8wekyb3d8bbwe";

        // Launch the retrieved file in 3D builder
        bool success = await Windows.System.Launcher.LaunchFileAsync(file,
options);

        // prompt the user to save as .3mf
        OutputTextBlock.Text = "save " + file.Name + " as a .3mf file and
reload.";

        // have user choose another file (ideally the newly-saved .3mf file)
        file = await openPicker.PickSingleFileAsync();

    } else {
        // if the file type is .3mf
        // notify user that load was successful
        OutputTextBlock.Text = file.Name + " loaded as file";
    }
}
```

Repair model data for 3D printing

Not all 3D model data is printable, even in the .3mf type. In order for the printer to correctly determine what space to fill and what to leave empty, each model to be printed must be a single seamless mesh, have outward-facing surface normals, and have manifold geometry. Issues in these areas can arise in a variety of different forms and can be hard to spot in complex shapes. However, modern software solutions are often adequate for converting raw geometry to printable 3D shapes. This is known as *repairing* the model and is implemented in the `OnFixClick` method shown here.

> ⓘ **Note**
>
> The 3D data file must be converted to implement IRandomAccessStream, which can then be used to generate a Printing3DModel object.

```
C#
```

```
private async void OnFixClick(object sender, RoutedEventArgs e) {

    // read the loaded file's data as a data stream
    IRandomAccessStream fileStream = await
file.OpenAsync(FileAccessMode.Read);

    // assign a Printing3DModel to this data stream
    Printing3DModel model = await
package.LoadModelFromPackageAsync(fileStream);

    // use Printing3DModel's repair function
    OutputTextBlock.Text = "repairing model";
    var data = model.RepairAsync();
```

The **Printing3DModel** object should now be repaired and printable. Use SaveModelToPackageAsync to assign the model to the **Printing3D3MFPackage** object that you declared when creating the class.

```
C#
```

```
    // save model to this class' Printing3D3MFPackage
    OutputTextBlock.Text = "saving model to 3MF package";
    await package.SaveModelToPackageAsync(model);

}
```

Execute printing task: create a TaskRequested handler

Later, when the 3D print dialog is displayed to the user and the user elects to begin printing, your app will need to pass in the desired parameters to the 3D print pipeline. The 3D print API will raise the TaskRequested event, which requires handling appropriately.

```
C#
```

```
private void MyTaskRequested(Print3DManager sender,
Print3DTaskRequestedEventArgs args) {
```

The core purpose of this method is to use the *args* parameter to send a **Printing3D3MFPackage** down the pipeline. The **Print3DTaskRequestedEventArgs** type has one property: Request. It is of the type Print3DTaskRequest and represents one print job request. Its method CreateTask lets the app submit the correct information for

your print job and return a reference to the **Print3DTask** object that was sent down the 3D print pipeline.

CreateTask has the following input parameters: a string for the print job name, a string for the ID of the printer to use, and a Print3DTaskSourceRequestedHandler delegate. The delegate is automatically invoked when the **3DTaskSourceRequested** event is raised (this is done by the API itself). The important thing to note is that this delegate is invoked when a print job is initiated, and it is responsible for providing the right 3D print package.

Print3DTaskSourceRequestedHandler takes one parameter, a Print3DTaskSourceRequestedArgs object, which contains the data to be sent. The SetSource method accepts the package to be printed. The following code shows a **Print3DTaskSourceRequestedHandler** delegate implementation (`sourceHandler`).

C#

```
// this delegate handles the API's request for a source package
Print3DTaskSourceRequestedHandler sourceHandler = delegate
(Print3DTaskSourceRequestedArgs sourceRequestedArgs) {
    sourceRequestedArgs.SetSource(package);
};
```

Next, call **CreateTask**, using the newly-defined delegate.

C#

```
// the Print3DTaskRequest ('Request'), a member of 'args', creates a
Print3DTask to be sent down the pipeline.
printTask = args.Request.CreateTask("Print Title", "Default",
sourceHandler);
```

The **Print3DTask** returned is assigned to the class variable declared in the beginning. This reference can be used to handle certain events thrown by the task.

C#

```
// optional events to handle
printTask.Completed += Task_Completed;
printTask.Submitting += Task_Submitting;
```

ⓘ Note

You must implement a `Task_Submitting` and `Task_Completed` method if you wish to register them to these events.

Execute printing task: open 3D print dialog

Finally, you need to launch the 3D print dialog that provides a number of printing options.

Here, we register a `MyTaskRequested` method with the **TaskRequested** event.

C#

```
private async void OnPrintClick(object sender, RoutedEventArgs e) {

    // get a reference to this class' Print3DManager
    Print3DManager myManager = Print3DManager.GetForCurrentView();

    // register the method 'MyTaskRequested' to the Print3DManager's
TaskRequested event
    myManager.TaskRequested += MyTaskRequested;
```

After registering the **TaskRequested** event handler, you can invoke the method **ShowPrintUIAsync**, which brings up the 3D print dialog in the current application window.

C#

```
// show the 3D print dialog
OutputTextBlock.Text = "opening print dialog";
var result = await Print3DManager.ShowPrintUIAsync();
```

It is also good practice to de-register your event handlers once your app resumes control.

C#

```
    // remove the print task request after dialog is shown
    myManager.TaskRequested -= MyTaskRequested;
}
```

Related topics

3D printing with Windows 10 Generate a 3MF package
3D printing UWP sample

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Generate a 3D Manufacturing Format package

Article • 05/06/2023

This guide describes the structure of the 3D Manufacturing Format (3MF) file type and how the **Windows.Graphics.Printing3D** API can be used to create and manipulate it.

Important APIs

- Windows.Graphics.Printing3D

What is 3D Manufacturing Format?

3MF is a set of conventions for using XML to describe the appearance and structure of 3D models for manufacturing (3D printing). It defines a set of parts (required and optional) and their relationships, to a 3D manufacturing device. A data set that adheres to the 3MF can be saved as a file with the .3mf extension.

The **Printing3D3MFPackage** class in the **Windows.Graphics.Printing3D** namespace is analogous to a single .3mf file, while other classes map to the particular XML elements in the .3mf file. This guide describes how each of the main parts of a 3MF document can be created and set programmatically, how the 3MF Materials Extension can be used, and how a **Printing3D3MFPackage** object can be converted and saved as a .3mf file. For more information on the standards of 3MF or the 3MF Materials Extension, see the 3MF Specification.

Core classes in the 3MF structure

The **Printing3D3MFPackage** class represents a complete 3MF document, and at the core of a 3MF document is its model part, represented by the **Printing3DModel** class. Most of the information about a 3D model is stored by setting the properties of the **Printing3DModel** class and the properties of their underlying classes.

C#

```
var localPackage = new Printing3D3MFPackage();
var model = new Printing3DModel();
// specify scaling units for model data
model.Unit = Printing3DModelUnit.Millimeter;
```

Metadata

The model part of a 3MF document can hold metadata in the form of key/value pairs of strings stored in the **Metadata** property. There is predefined metadata, but custom pairs can be added as part of an extension (described in more detail in the 3MF specification). It is up to the receiver of the package (a 3D manufacturing device) to determine whether and how to handle metadata, but it is good practice to include as much info as possible in the 3MF package.

C#

```
model.Metadata.Add("Title", "Cube");
model.Metadata.Add("Designer", "John Smith");
model.Metadata.Add("CreationDate", "1/1/2016");
```

Mesh data

In this guide, a mesh is a body of 3-dimensional geometry constructed from a single set of vertices (though it does not have to appear as a single solid). A mesh part is represented by the **Printing3DMesh** class. A valid mesh object must contain information about the location of all vertices as well as all triangle faces that exist between certain sets of vertices.

The following method adds vertices to a mesh and then gives them locations in 3D space.

C#

```
private async Task GetVerticesAsync(Printing3DMesh mesh) {
    Printing3DBufferDescription description;

    description.Format = Printing3DBufferFormat.Printing3DDouble;

    // have 3 xyz values
    description.Stride = 3;

    // have 8 vertices in all in this mesh
    mesh.CreateVertexPositions(sizeof(double) * 3 * 8);
    mesh.VertexPositionsDescription = description;

    // set the locations (in 3D coordinate space) of each vertex
    using (var stream = mesh.GetVertexPositions().AsStream()) {
        double[] vertices =
        {
            0, 0, 0,
            10, 0, 0,
```